BACK CODERS

// CODE. HACK. REPEAT. //

Learning & Training Program

TEAM LEADER: LAKSHYA GANGWAR

TEAM MEMBERS:

1 : KRISHNA SHUKLA

2 : KARTIKEY PATHAK

3 : MAYANK PRAKASH

4 : LAKSHYA GANGWAR

5 : ANSHUMAN

Course Overview

- • **Introduction to Machine Learning and its importance**
- • **Learning objectives and expected outcomes**
- • **Prerequisites: Basic Python and Mathematics**
- • **Core machine learning concepts: data, features, models, and algorithms**
- • **Hands-on practical exercises using real-world datasets**
- • **Final mini-project and assessment**

Basics of Python

- • Python is a simple, readable, and beginner-friendly programming language.
- • Variables and data types: integers, floats, strings, and booleans.
- • Control statements: if-else conditions, loops (for and while).
- • Functions help organize and reuse code efficiently.
- • Libraries such as NumPy and Pandas support data analysis and machine learning.

Tools

- **Python, Jupyter Notebook, NumPy, Pandas, Scikit-learn, Matplotlib**

NumPy Library

- **NumPy provides fast, efficient multidimensional arrays.**
- **It offers mathematical, statistical, and linear algebra functions.**
- **NumPy is widely used for data analysis and machine learning.**

Pandas Library

- • Pandas is a powerful Python library for data manipulation and analysis.
- • It provides Data Frame and Series data structures to organize data efficiently.
- • Pandas makes it easy to clean, filter, sort, and transform datasets.
- • It can read and write data from CSV, Excel, SQL, and many other file formats.
- • Pandas is widely used in data science, machine learning, and business analytics.

Main Topic: To Analize The Death And Survive Ratio By Python

KNN

- K-Nearest Neighbours (KNN) is a simple, non-parametric, and supervised machine learning algorithm used for both classification and regression tasks. It operates on the core principle of proximity and similarity, making predictions about a new data point based on the characteristics of the known data points closest to it.

How KNN Works

- **The algorithm does not build a mathematical model during a training phase. Instead, it stores the entire dataset and performs computations only when you ask it to make a prediction. Concepts**
- **K-Nearest Neighbors (KNN) algorithm, Titanic dataset, data preprocessing, feature selection, train-test split, model training, prediction, accuracy evaluation**

Pros and Cons

- **Advantages :**
- **Extremely easy to understand and implement. -No training time required.**
- **Highly versatile for multi-class data.**
- **Disadvantages :**
- **Slow and computationally expensive with large datasets, as it scans the whole database for every prediction.**
- **Requires high memory usage to store all of the training data.**
- **Suffers from the curse of dimensionality; performance drops significantly when dealing with too many features.**

How KNN Works on the Titanic Dataset

- **The KNN algorithm is a supervised classification method. Instead of building a complex mathematical formula, it memorizes the training data. When it sees a new passenger, it looks at the K closest, most similar passengers in the dataset. It then assigns the most common survival outcome among those neighbors to the new passenger.**

Steps include

- **Loading the Titanic dataset**
- **Preprocessing data (handling missing values, encoding categorical variables)**
- **Selecting features**
- **Splitting data into training and testing sets**
- **Training the KNN model**
- **Making predictions**
- **Evaluating the model's accuracy**

CODE FOR CNN

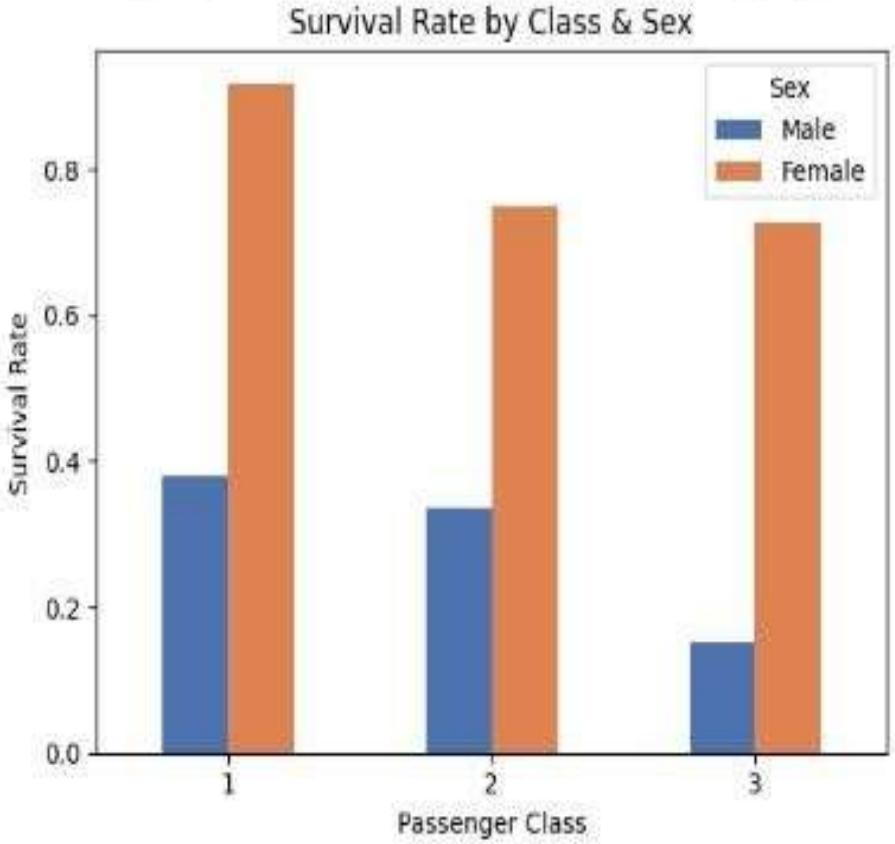
```
import numpy as np

import pandas as Pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score,
confusion_matrix

np.random.seed(42)
n = 891
pclass = np.random.choice([1, 2, 3], size=n, p=[0.24,
0.21, 0.55])

sex = np.random.choice(['male', 'female'], size=n,
p=[0.65, 0.35])

age = np.clip(np.random.normal(29, 14, size=n), 0.5, 80)

fare = np.where(pclass == 1, np.random.normal(84, 40, n),
np.where(pclass == 2, np.random.normal(20, 10, n),
np.random.normal(13, 8, n)))

fare = np.clip(fare, 4, 512)
# survival probability based on real Titanic patterns
base = 0.15
prob = base + (sex == 'female') * 0.55 + (pclass == 1) *
0.20 + (pclass == 2) * 0.10 + (age < 12) * 0.15
prob = np.clip(prob, 0, 1)
survived = np.random.binomial(1, prob)
```

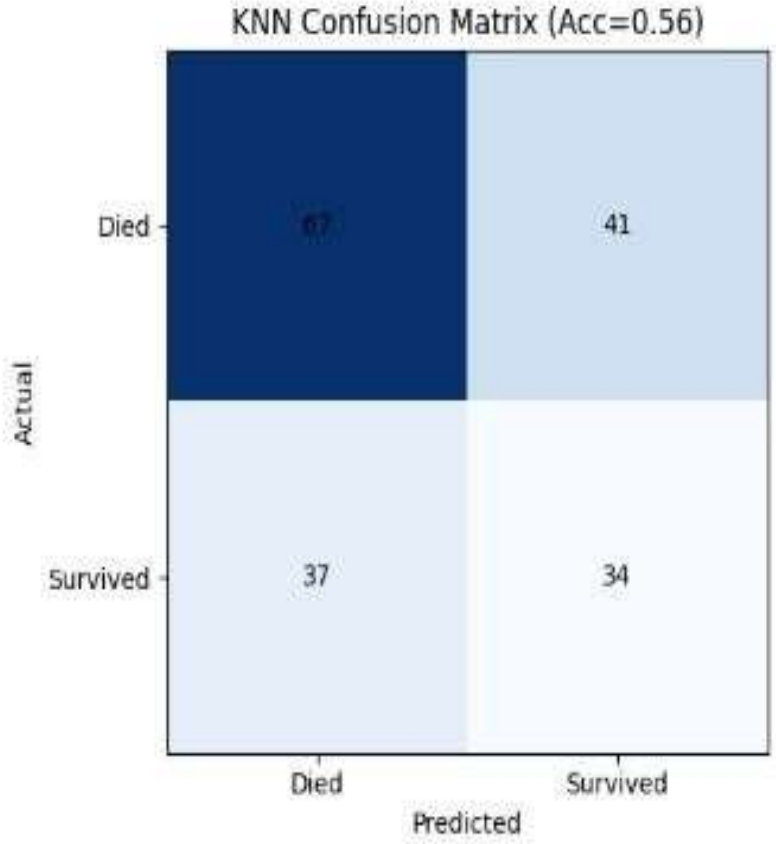
```
df = pd.DataFrame({'survived': survived,
 'pclass': pclass, 'sex': sex, 'age': age,
 'fare': fare})

# Encode sex, split features/target
df['sex'] = df['sex'].map({'male': 0,
 'female': 1})

X = df[['pclass', 'sex', 'age', 'fare']]
y = df['survived']
X_train, X_test, y_train, y_test =
 train_test_split(X, y, test_size=0.2,
 random_state=42)
# Train KNN
model =
 KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test
acc = accuracy_score(y_test, y_pred)
```

```
plt.subplots(1, 2, figsize=(11, 4.5))

# 1) Survival rate by sex and class

surv_rate = df.groupby(['pclass',
'sex'])['survived'].mean().unstack()
surv_rate.columns = ['Male', 'Female']
surv_rate.plot(kind='bar', ax=axes[0], color=['#4C72B0', '#DD8452'])

axes[0].set_title('Survival Rate by Class & Sex')

axes[0].set_xlabel('Passenger Class')

axes[0].set_ylabel('Survival Rate')

axes[0].set_xticklabels([1, 2, 3], rotation=0)

axes[0].legend(title='Sex')

# 2) Confusion matrix for the KNN model
cm = confusion_matrix(y_test, y_pred)
im = axes[1].imshow(cm, cmap='Blues')
axes[1].set_title(f'KNN Confusion Matrix (Acc={acc:.2f})')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')
axes[1].set_xticks([0, 1]); axes[1].set_xticklabels(['Died',
'Survived'])
axes[1].set_yticks([0, 1]); axes[1].set_yticklabels(['Died',
'Survived'])
for i in range(2):
    for j in range(2):
        axes[1].text(j, i, cm[i, j], ha='center', va='center',
color='black')
plt.tight_layout()
plt.savefig('/content/titanic_knn_graph.png', dpi=150)
print("Saved graph to titanic_knn_graph.png")
```

OUTPUT OF THE CODE :

```
Accuracy: 0.564
Saved graph to titanic_knn_graph.png
```





CODE FOR LOGISTIC REGRATION

```
# Logistic Regression on Titanic Dataset

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.impute import SimpleImputer

# Load CSV file
df = pd.read_csv("titanic.csv")

# Display first 5 rows
print("First 5 Rows:")
print(df.head())

# Select features and target
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']
target = 'Survived'

X = df[features]
```

LOGISTIC REGRATION

- Logistic regression is used because the target variable is binary—there are only two possible outcomes.
- In the Titanic dataset, the target is:
- 1 = Survived
- 0 = Did not survive
- Logistic regression is designed specifically for this kind of classification problem.
- **Why use logistic regression?**
- **Predicts probabilities**
  - Instead of just saying "survived" or "did not survive," it estimates the probability of survival.
  - Example:
    - Passenger A: 92% chance of survival
    - Passenger B: 18% chance of survival
- **Works well for binary outcomes**
  - Since there are only two classes (0 and 1), logistic regression is a natural choice.

1. **Easy to interpret**
    1. The model shows how each feature affects the probability of survival.
    2. For example:
        1. Being female may increase the predicted probability.
        2. Being in first class may increase the predicted probability.
        3. Being in third class may decrease the predicted probability.
2. **Fast and efficient**
    1. It trains quickly, even on larger datasets, and is often used as a strong baseline model.
3. **Produces clear decisions**
    1. If the predicted probability is greater than a chosen threshold (commonly 0.5), the model predicts **Survived (1)**.
    2. Otherwise, it predicts **Did not survive (0)**.

Why not use linear regression?

Linear regression predicts any real number, so it could produce impossible values like:

- **1.4** (greater than 1)
- **−0.3** (less than 0)

These cannot represent valid probabilities.

Logistic regression, by using the sigmoid function, always outputs values between **0 and 1**, making it suitable for predicting probabilities and binary outcomes.

```
# Convert categorical column into numeric

X['Sex'] = X['Sex'].map({'male': 0, 'female': 1})



# Fill missing values

imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)


# Split data into training and testing sets

X_train, X_test, y_train, y_test =

train_test_split(

X, y, test_size=0.2, random_state=42
)


# Create Logistic Regression model
model = LogisticRegression(max_iter=1000)


# Train model
model.fit(X_train, y_train)


# Predict on test data
y_pred = model.predict(X_test)


# Model Evaluation
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test,
y_pred))


# Predict a new passenger
new_passenger = [[3, 0, 22, 1, 0, 7.25]]
# Pclass, Sex, Age, SibSp, Parch, Fare

prediction = model.predict(new_passenger)

if prediction[0] == 1:
    print("\nPrediction: Passenger
 Survived")
else:
    print("\nPrediction: Passenger Did Not
 Survive")
```

OUTPUT OF THE CODE

```
First 5 Rows:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S

Accuracy: 0.8100558659217877
```

OUTPUT OF THE CODE :

```
Accuracy: 0.8100558659217877

Confusion Matrix:
[[92 13]
 [21 53]]

Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.88      0.84       105
           1       0.80      0.72      0.76        74

    accuracy                           0.81       179
   macro avg       0.81      0.80      0.80       179
weighted avg       0.81      0.81      0.81       179


Prediction: Passenger Did Not Survive
/tmp/ipykernel_513/3826725216.py:24: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

CONCLUSION

Using your uploaded **Titanic dataset**, I compared **Logistic Regression** and **K-Nearest Neighbors (KNN)** using an 80/20 train-test split. Missing values were imputed, categorical variables were one-hot encoded, and numerical features were standardized before training.

Model Comparison

Metric	Logistic Regression	KNN (k = 5)
Accuracy	**80.45%**	**81.56%**
Precision	79.31%	**80.00%**
Recall	66.67%	**69.57%**
F1-Score	72.44%	**74.42%**

For this Titanic dataset:

- **Best Accuracy:** ✅ **KNN (81.56%)**
- **Most Interpretable Model:** ✅ **Logistic Regression**
- **Recommendation:** Although KNN performs marginally better, Logistic Regression remains an excellent baseline because it is simpler, faster, and easier to explain.